

GPUMODE Lecture Study Notes: Lecture 78 Iris: Multi-GPU Programming in Triton

Core Problem the Lecture Addresses

This lecture presents **Iris**, an open-source Triton-based framework for first-class multi-GPU programming. The central problem is that mainstream multi-GPU programming is still largely organized around a **bulk-synchronous** execution model. In that model, the host launches a compute kernel, waits for it to finish, launches a communication operation, waits for communication to finish, and then launches the next piece of work.

The bulk-synchronous sequence is:

compute $\longrightarrow$ global completion $\longrightarrow$ communication $\longrightarrow$ global completion $\longrightarrow$ more compute.

This is simple, portable, and compatible with collective libraries, but it introduces avoidable costs:

- communication is usually initiated by the CPU rather than by GPU workgroups;
- there are full-kernel barriers between phases;
- intermediate global-memory buffers are often required;
- compute and communication are difficult to overlap at fine granularity;
- the programmer loses control over which workgroups communicate and when.

Iris tries to replace this stop-and-go model with a **fine-grained device-side model**. In Iris, a Triton kernel can compute a tile, directly store that tile into remote GPU memory, use atomics to signal readiness, and allow another GPU or another workgroup to consume the data without ending the kernel.

The target programming model is:

workgroup compute $\longrightarrow$ remote store or load $\longrightarrow$ device-side synchronization $\longrightarrow$ more work.

The essential thesis is:

Multi-GPU communication should be expressible inside the same Triton kernel abstraction that already expresses GPU compute. A remote load, remote store, put, get, or atomic should feel like a natural extension of Triton’s device-side memory model, not like a separate host-side collective operation.

Required Background

Triton Execution Model

Triton programs are written as Python functions decorated with JIT compilation. Each program instance is conceptually similar to a CUDA thread block or GPU workgroup. A Triton program typically computes one tile of an output tensor.

For a matrix multiplication $C = AB$, a persistent Triton GEMM kernel might assign each program instance to a tile:

$$C_{m:n} = \sum_k A_{m:k} B_{k:n}.$$

A simplified mapping is:

program id $\longrightarrow$ (m -tile, n -tile) $\longrightarrow$ accumulator registers $\longrightarrow$ global memory store.

The Triton programmer controls:

- tile sizes such as B_M , B_N , and B_K ;
- pointer arithmetic;
- vectorized loads and stores;
- masks for boundary tiles;
- caching modifiers;
- persistent scheduling strategies.

Iris extends this model so that the pointer arithmetic may refer not only to local GPU memory but also to another GPU’s memory.

GPU Memory Hierarchy

A modern GPU exposes several levels of memory:

- **registers**: private to each thread or lane, fastest, limited capacity;
- **shared memory or LDS**: explicit on-chip storage shared within a block or workgroup;
- **L1 cache**: close to the compute unit or SM;
- **L2 cache**: shared across larger parts of the GPU;
- **HBM**: high-bandwidth device memory;
- **interconnect memory path**: access to remote GPU memory over links such as XGMI.

A rough locality ordering is:

registers < shared memory < L1 < L2 < HBM < remote HBM over XGMI < remote memory over RDMA.

The symbol < means “lower latency and usually cheaper access than.” Iris explicitly exposes this cost hierarchy by letting the programmer decide when to use remote memory accesses and when to synchronize.

Bulk-Synchronous Communication

Traditional collective programming often uses calls such as all-reduce, all-gather, reduce-scatter, or all-to-all as large operations. A GEMM plus communication pattern may look like:

$$C_i = AB_i, \quad \{C_i\}_{i=0}^{P-1} \xrightarrow{\text{all-gather or all-scatter}} \text{visible to all ranks.}$$

Here P is the number of GPUs or ranks. The standard implementation sequence is:

$$T_{\text{total}} = T_{\text{GEMM}} + T_{\text{barrier}} + T_{\text{comm}} + T_{\text{barrier}}.$$

If communication and computation can overlap, the idealized time becomes closer to:

$$T_{\text{overlap}} \approx \max(T_{\text{GEMM}}, T_{\text{comm}}) + T_{\text{sync-overhead}}.$$

The purpose of Iris is to reduce $T_{\text{sync-overhead}}$ and allow overlap at tile granularity.

Partitioned Global Address Space

Iris is built around a **symmetric heap**, which is a partitioned global address space abstraction. Each GPU rank owns a local heap allocation, and corresponding tensors are allocated at the same offset inside each rank's heap.

Let:

$$H_r$$

be the base virtual address of the symmetric heap on rank r . Let a local pointer on rank a be:

$$p_a = H_a + \Delta.$$

The same logical object on rank b is found at:

$$p_b = H_b + \Delta.$$

Thus the translation from a pointer in the current rank's heap to the corresponding pointer in another rank's heap is:

$$\text{translate}(p_a, a, b) = H_b + (p_a - H_a).$$

This tiny equation is the heart of the Iris device-side backend.

Main Concepts

What Iris Provides

Iris provides a small Triton-native framework for remote memory access. Its main design goals are:

- make multi-GPU programming feel like normal Triton programming;
- expose device-side remote loads, stores, puts, gets, and atomics;
- avoid forcing every communication pattern into a host-side collective;
- allow fine-grained compute and communication overlap;
- preserve low-level control for expert GPU programmers.

The lecture emphasizes that Iris is not primarily trying to replace collective libraries at the host API level. Instead, it provides the low-level device-side primitives from which custom collectives and fused compute-communication kernels can be built.

PyTorch-Like Host-Side Allocation

On the host side, Iris exposes tensor creation APIs that resemble PyTorch. Examples include operations analogous to:

- `full`;
- `zeros`;
- `ones`;
- `zeros_like`;
- `arange`;
- `linspace`.

The important distinction is that tensors allocated through Iris are placed inside the symmetric heap. They are still ordinary PyTorch tensors from the user’s perspective, but their storage is suitable for remote memory address translation.

The invariant is:

remote access is valid $\iff$ the tensor lives in the symmetric heap.

This leads to a practical warning:

If a PyTorch operation reallocates a tensor outside the Iris symmetric heap, Iris can no longer translate that pointer for remote memory access. Use in-place operations when preserving symmetric-heap placement matters.

For example, an in-place update preserves storage:

```
x.add_(1.0)      # safe: preserves the same allocation
```

A new allocation may break the Iris invariant:

```
x = x + 1.0      # risky: may allocate outside the symmetric heap
```

Triton-Like Device-Side APIs

On the device side, Iris provides APIs that resemble Triton’s memory operations. A remote load has the same spirit as `tl.load`, but it also needs enough information to compute a remote address.

Conceptually:

```
value = iris.load(  
    ptr,  
    from_rank=current_rank,  
    to_rank=remote_rank,
```

```

    heap_bases=heap_bases
)

```

A remote store is similarly:

```

iris.store(
    ptr,
    value,
    from_rank=current_rank,
    to_rank=remote_rank,
    heap_bases=heap_bases
)

```

The extra arguments exist because remote pointer translation requires:

$$\{H_0, H_1, \dots, H_{P-1}\},$$

the heap base addresses for all ranks. These heap bases are communicated once during initialization, often using a standard distributed setup.

Symmetric Heap Translation

The lecture repeatedly emphasizes that most Iris operations are simply:

$$\text{address translation} \longrightarrow \text{ordinary memory operation.}$$

A simplified device-side translation is:

```

@triton.jit
def translate(ptr, from_rank, to_rank, heap_bases):
    from_base = tl.load(heap_bases + from_rank)
    to_base = tl.load(heap_bases + to_rank)

    offset = ptr - from_base
    remote_ptr = to_base + offset

    return remote_ptr

```

A remote store can then be understood as:

```

@triton.jit
def iris_store(ptr, value, from_rank, to_rank, heap_bases):
    remote_ptr = translate(ptr, from_rank, to_rank, heap_bases)
    tl.store(remote_ptr, value)

```

A remote load is:

```

@triton.jit
def iris_load(ptr, from_rank, to_rank, heap_bases):

```

```
remote_ptr = translate(ptr, from_rank, to_rank, heap_bases)
return tl.load(remote_ptr)
```

The address arithmetic is:

$$\Delta = p - H_{\text{from}},$$

$$p_{\text{remote}} = H_{\text{to}} + \Delta.$$

For example, if:

$$p - H_{\text{from}} = 448,$$

then the corresponding remote address on rank r is:

$$p_r = H_r + 448.$$

This is why Iris can be small: the core idea is simple pointer arithmetic plus Triton-generated memory instructions.

Remote Memory Access

Remote memory access means one GPU issues a load or store targeting memory physically attached to another GPU. On AMD systems, intra-node GPU-to-GPU traffic may travel over XGMI.

For a point-to-point remote store:

$$\text{GPU } i \xrightarrow{\text{store over XGMI}} \text{HBM of GPU } j.$$

For a point-to-point remote load:

$$\text{GPU } i \xleftarrow{\text{load over XGMI}} \text{HBM of GPU } j.$$

The lecture reports that Iris microbenchmarks achieved bandwidth close to hardware peak for both local HBM and remote XGMI paths. The diagonal of the benchmark matrix represents local access:

$$i = j \quad \Rightarrow \quad \text{local HBM bandwidth.}$$

The off-diagonal entries represent remote access:

$$i \neq j \quad \Rightarrow \quad \text{XGMI bandwidth.}$$

The performance target is:

$$\frac{B_{\text{Iris}}}{B_{\text{hardware peak}}} \approx 1.$$

The point is not only the measured percentage but also that the generated assembly is inspectable. Because Iris is Triton-native, the resulting remote access code is inside the generated kernel rather than hidden behind an external opaque runtime call.

Hardware-Level Explanation

Why Kernel Boundaries Synchronize Coarsely

When a GPU kernel ends, the hardware and runtime enforce visibility guarantees for subsequent work in the stream. Data produced by the completed kernel is made visible to later kernels. In a coarse host-driven model, this gives an ordering like:

$$K_1 \prec K_2,$$

where K_2 sees memory effects from K_1 if both are ordered in the same stream.

This is convenient but expensive because it imposes a full-kernel phase boundary. The programmer cannot say:

communicate tile t as soon as tile t is ready.

Instead, the communication phase often waits for all tiles:

communicate all tiles after every tile is complete.

Iris moves the synchronization point from kernel granularity to tile or workgroup granularity.

Why Atomics Are Necessary

Remote stores alone do not establish a safe producer-consumer relationship. A producer may write data and then write a flag. Without memory-ordering constraints, the compiler or hardware may reorder or delay visibility of the data store relative to the flag store.

The desired happens-before relationship is:

producer writes data $\rightarrow$ producer releases flag $\rightarrow$ consumer acquires flag $\rightarrow$ consumer reads data.

Iris uses atomic operations with memory order and scope to enforce this. The producer uses release semantics:

$$\text{atomic_cas}_{\text{release, system}}(\text{flag}, 0, 1).$$

The consumer uses acquire semantics:

$$\text{atomic_cas}_{\text{acquire, system}}(\text{flag}, 1, 1).$$

Release means that previous writes cannot be reordered after the release operation. Acquire means that later reads cannot be reordered before the acquire operation.

Together:

$$\text{release} + \text{acquire} \Rightarrow \text{consumer observes the producer's data.}$$

Memory Scopes

The lecture discusses memory scopes as a clean way to express where synchronization must be visible. A simplified ordering of scopes is:

$$\text{wavefront} < \text{workgroup} < \text{GPU} < \text{system} < \text{world}.$$

The larger the scope, the more expensive the synchronization may be. For intra-workgroup communication, a workgroup scope may suffice. For communication across different GPUs in the same node, system scope is appropriate. For future scale-out across nodes, Iris aims to support a world scope.

A useful mental model is:

$$\text{larger synchronization scope} \Rightarrow \text{larger visibility domain and higher cost.}$$

Cache Visibility and Cache Modifiers

The lecture emphasizes that explicit cache behavior matters for multi-GPU kernels. If a GPU writes data that will only be consumed by another GPU, caching that data locally may be wasteful. It can pollute L1 or L2 cache and evict more useful data.

For producer-only communication data, the desired policy may be:

write data $\rightarrow$ make visible remotely $\rightarrow$ avoid local cache pollution.

Triton-style cache modifiers can express policies such as write-through or non-temporal behavior. Conceptually:

```
tl.store(ptr, value, cache_modifier=".wt")
```

In communication-heavy kernels, cache modifiers can matter because communication tiles may not be reused by the producer. In GEMM-heavy kernels, cache modifiers can also help when one operand has high reuse and another has little reuse.

For a tall-skinny GEMM, suppose:

$$A \in \mathbb{R}^{M \times K}, \quad B \in \mathbb{R}^{K \times N},$$

with K small and M very large. One operand may fit well in cache, while another streams through memory. The programmer may want:

cache reused operand do not cache streaming operand.

Iris inherits this style of low-level control because remote memory operations are expressed inside Triton.

Visual Concept

Draw a GPU node with multiple GPUs connected by XGMI. Inside each GPU, show HBM, L2, L1, compute units or SMs, and workgroups. Then draw a remote store from a workgroup on GPU 0 into HBM on GPU 1, followed by a system-scope atomic flag.

Mathematical Reasoning

Symmetric Heap Address Translation

Let P be the number of ranks. Each rank r has a heap base:

$$H_r \in \mathbb{N}.$$

A tensor X is allocated at the same logical offset Δ_X in every heap:

$$X_r = H_r + \Delta_X.$$

If the current rank is a and has pointer:

$$p_a = H_a + \Delta_X,$$

then the remote pointer on rank b is:

$$p_b = H_b + (p_a - H_a).$$

Thus Iris remote pointer translation is:

$$\boxed{\text{translate}(p, a, b) = H_b + p - H_a}$$

This formula is the main reason that allocation must happen in the symmetric heap. If a tensor is allocated outside the heap, then $p - H_a$ is not a meaningful symmetric offset.

Bandwidth Efficiency

For a remote memory operation, define achieved bandwidth as:

$$B_{\text{achieved}} = \frac{\text{bytes transferred}}{\text{elapsed time}}.$$

A normalized efficiency metric is:

$$\eta = \frac{B_{\text{achieved}}}{B_{\text{peak}}}.$$

For local HBM access:

$$B_{\text{peak}} = B_{\text{HBM}}.$$

For remote XGMI access:

$$B_{\text{peak}} = B_{\text{XGMI}}.$$

The benchmark goal is:

$$\eta \approx 1.$$

If η is close to 1, then the abstraction overhead is small. If η is much smaller than 1, possible causes include insufficient concurrency, poor memory access pattern, address-calculation overhead, poor loop unrolling, cache effects, or interconnect saturation issues.

Compute and Communication Overlap

Let:

$$T_c = \text{time for computation}, \quad T_m = \text{time for communication}.$$

In a bulk-synchronous model:

$$T_{\text{bulk}} = T_c + T_m + T_{\text{barrier}}.$$

With perfect overlap:

$$T_{\text{ideal}} = \max(T_c, T_m).$$

With real overlap:

$$T_{\text{real}} = \max(T_c, T_m) + T_{\text{lock}} + T_{\text{imbalance}} + T_{\text{tail}}.$$

Here:

- T_{lock} is the cost of atomics and spinning;
- $T_{\text{imbalance}}$ is the cost of assigning too many or too few resources to communication;
- T_{tail} is the underutilization at the end of the schedule.

The speedup over bulk synchronous execution is:

$$S = \frac{T_{\text{bulk}}}{T_{\text{real}}}.$$

If $T_c \approx T_m$ and overheads are small, overlap can approach:

$$S \approx \frac{T_c + T_m}{\max(T_c, T_m)} \approx 2.$$

If one phase dominates, the maximum possible overlap benefit is smaller.

Resource Partitioning

Suppose a GPU has N_{SM} compute units or SMs. A producer-consumer design may allocate:

$$N_{\text{GEMM}}$$

SMs to compute and:

$$N_{\text{comm}} = N_{\text{SM}} - N_{\text{GEMM}}$$

SMs to communication. The throughput balance condition is roughly:

$$\frac{\text{tiles produced per second}}{\text{tiles consumed per second}} \approx 1.$$

If GEMM produces tiles faster than communication consumes them, the communication side bottlenecks:

$$R_{\text{GEMM}} > R_{\text{comm}} \quad \Rightarrow \quad \text{flags accumulate and communication lags.}$$

If communication consumes faster than GEMM produces, communication work-groups spin:

$$R_{\text{comm}} > R_{\text{GEMM}} \quad \Rightarrow \quad \text{communication resources are underutilized.}$$

The tuning problem is:

$$\min_{N_{\text{GEMM}}} T_{\text{real}}(N_{\text{GEMM}}, N_{\text{SM}} - N_{\text{GEMM}}).$$

The lecture notes that this is not fully solved. Possible approaches include manual tuning, analytical modeling, autotuning for constrained shape sets, or work-queue-based dynamic scheduling.

Arithmetic Intensity

For GEMM:

$$C = AB,$$

where:

$$A \in \mathbb{R}^{M \times K}, \quad B \in \mathbb{R}^{K \times N}, \quad C \in \mathbb{R}^{M \times N},$$

the floating-point work is approximately:

$$\text{FLOPs} = 2MNK.$$

If each element has s bytes and each matrix is read or written once, the minimum data movement is approximately:

$$\text{Bytes} = s(MK + KN + MN).$$

The arithmetic intensity is:

$$I = \frac{2MNK}{s(MK + KN + MN)}$$

A high I means GEMM is compute-heavy and can often hide communication. A low I means communication and memory bandwidth are more likely to dominate.

Code Walkthrough

Host-Side Setup

A simplified Iris program begins by initializing distributed state, constructing an Iris context, and allocating symmetric-heap tensors.

```
import torch
import torch.distributed as dist
import triton
import triton.language as tl
import iris

dist.init_process_group(backend="nccl")

rank = dist.get_rank()
world_size = dist.get_world_size()

ctx = iris.Iris()

x = iris.full((1024,), fill_value=rank, device="cuda")
flags = iris.zeros((world_size,), dtype=torch.int32, device="cuda")

heap_bases = ctx.heap_bases()
```

The important points are:

- `dist.init_process_group` establishes initial process communication;

- `iris.Iris()` constructs the symmetric heap and shares heap bases;
- `iris.full` and `iris.zeros` return PyTorch-like tensors backed by symmetric-heap memory;
- `heap_bases` is passed to kernels so device-side code can translate addresses.

The invariant is:

$$x_r = H_r + \Delta_x$$

for every rank r .

Producer Kernel

A producer computes data, stores it to a remote rank, and then releases a flag.

```
@triton.jit
def producer_kernel(x_ptr, flag_ptr, heap_bases, rank: tl.constexpr):
    offsets = tl.arange(0, 1024)

    values = offsets.to(tl.float32)

    remote_rank = 1

    iris.store(
        x_ptr + offsets,
        values,
        from_rank=rank,
        to_rank=remote_rank,
        heap_bases=heap_bases
    )

    iris.atomic_cas(
        flag_ptr,
        0,
        1,
        sem="release",
        scope="system",
        from_rank=rank,
        to_rank=remote_rank,
        heap_bases=heap_bases
    )
```

Line-by-line hardware interpretation:

- `offsets` creates a vector of element indices inside one Triton program.
- `values` represents produced data, normally the output of real computation.

- `iris.store` translates the local symmetric-heap pointer to the corresponding remote address on rank 1 and emits a store.
- `iris.atomic_cas` updates a remote flag using release semantics.
- `scope="system"` makes the synchronization visible across GPUs in the system.

The required ordering is:

remote data store $\prec$ release flag.

Consumer Kernel

The consumer waits until the producer's flag is visible, then loads the data.

```
@triton.jit
def consumer_kernel(x_ptr, flag_ptr, out_ptr, heap_bases, rank: tl.constexpr):
    producer_rank = 0

    old = tl.full((), 0, tl.int32)

    while old != 1:
        old = iris.atomic_cas(
            flag_ptr,
            1,
            1,
            sem="acquire",
            scope="system",
            from_rank=rank,
            to_rank=producer_rank,
            heap_bases=heap_bases
        )

    offsets = tl.arange(0, 1024)

    values = iris.load(
        x_ptr + offsets,
        from_rank=rank,
        to_rank=producer_rank,
        heap_bases=heap_bases
    )

    tl.store(out_ptr + offsets, values)
```

The consumer's spin loop is a device-side wait. Once it observes the flag with acquire semantics, it is safe to read the remote data.

The ordering is:

acquire flag $\prec$ remote data load.

Together with the producer:

producer data store $\prec$ producer release $\prec$ consumer acquire $\prec$ consumer data load.

Unfused Bulk-Synchronous GEMM Plus All-Scatter

The simplest GEMM plus communication design uses two separate kernels. The first computes GEMM tiles and writes them locally. The second scatters those tiles to other ranks.

```
# Kernel 1: persistent GEMM
persistent_gemm[(num_gemm_programs,)](
    A, B, C,
    M, N, K,
    BLOCK_M, BLOCK_N, BLOCK_K,
    num_sms=total_sms,
    stream=main_stream
)

# Kernel 2: persistent all-scatter
persistent_all_scatter[(num_comm_programs,)](
    C, C_remote,
    heap_bases,
    rank,
    world_size,
    num_sms=total_sms,
    stream=main_stream
)
```

Because both kernels are launched on the same stream, the communication kernel starts after GEMM completes. This means explicit device-side flags are not necessary for correctness.

The cost is:

$$T_{\text{bulk}} = T_{\text{GEMM}} + T_{\text{all-scatter}}.$$

The benefit is simplicity. The drawback is that communication cannot begin when the first tile is ready.

Producer-Consumer with Separate Kernels

A better design launches a GEMM producer kernel and a communication consumer kernel concurrently on different streams. The GPU resources are partitioned.

```
gemm_sms = 256
comm_sms = total_sms - gemm_sms

persistent_gemm_producer[(gemm_sms,)](
    A, B, C,
    flags,
    heap_bases,
    rank,
    stream=compute_stream
)

persistent_all_scatter_consumer[(comm_sms,)](
    C,
    flags,
    heap_bases,
    rank,
    world_size,
    stream=comm_stream
)
```

The producer computes a tile, writes it to global memory, and releases a per-tile flag. The consumer waits for each flag, acquires it, and scatters the corresponding tile.

Conceptually:

produce tile $t \rightarrow$ release flag $f_t \rightarrow$ consume tile $t \rightarrow$ remote scatter.

This gives overlap:

$$T_{\text{producer-consumer}} \approx \max(T_{\text{GEMM}}, T_{\text{comm}}) + T_{\text{flags}}.$$

The tuning challenge is choosing N_{GEMM} and N_{comm} .

Sequentially Fused GEMM Plus Communication

The next design fuses GEMM and communication into one kernel. Each work-group computes a GEMM tile and then immediately scatters the tile.

```

@triton.jit
def fused_gemm_all_scatter(A, B, C, heap_bases, rank: tl.constexpr,
                           world_size: tl.constexpr):
    pid = tl.program_id(0)

    acc = compute_gemm_tile(A, B, pid)

    for remote_rank in range(world_size):
        if remote_rank == rank:
            tl.store(local_c_pointer(C, pid), acc)
        else:
            iris.store(
                local_c_pointer(C, pid),
                acc,
                from_rank=rank,
                to_rank=remote_rank,
                heap_bases=heap_bases
            )

```

The main advantage is that the tile can be communicated while still in registers:

accumulator registers $\rightarrow$ remote store.

This can avoid a local HBM round trip:

register $\rightarrow$ HBM $\rightarrow$ load $\rightarrow$ remote store.

However, this design may increase register pressure and worsen tail effects. If one workgroup has extra communication work, other compute units may become idle near the end of the kernel.

Workgroup-Specialized Fused Kernel

The lecture's most interesting pattern is a single kernel with workgroup specialization. Some workgroups perform GEMM, while others perform communication.

```

@triton.jit
def specialized_gemm_all_scatter(A, B, C, flags, heap_bases,
                                 rank: tl.constexpr,
                                 world_size: tl.constexpr,
                                 gemm_sms: tl.constexpr):
    pid = tl.program_id(0)

    if pid < gemm_sms:
        tile_id = compute_tile_id(pid)

```

```

acc = compute_gemm_tile(A, B, tile_id)

tl.store(local_c_pointer(C, tile_id), acc, cache_modifier=".wt")

release_tile_flag(flags, tile_id)

else:
    comm_pid = pid - gemm_sms

    tile_id = get_comm_tile_id(comm_pid)

    wait_for_tile_flag(flags, tile_id)

    tile = tl.load(local_c_pointer(C, tile_id))

    for remote_rank in range(world_size):
        if remote_rank != rank:
            iris.store(
                local_c_pointer(C, tile_id),
                tile,
                from_rank=rank,
                to_rank=remote_rank,
                heap_bases=heap_bases
            )

```

This kernel uses one launch, but inside the kernel there are two roles:

program id < N_{GEMM} $\Rightarrow$ compute role,

program id $\geq N_{\text{GEMM}}$ $\Rightarrow$ communication role.

The benefit is that compute and communication are co-resident in one kernel. The cost is that the programmer must manage work partitioning and synchronization explicitly.

Remote All-Scatter Logic

For a GEMM plus all-scatter pattern, suppose B is column-sharded:

$$B = [B_0 \ B_1 \ \cdots \ B_{P-1}].$$

Each rank r computes:

$$C_r = AB_r.$$

An all-scatter or all-gather-like operation makes all C_r tiles visible to all ranks. For each produced tile $C_r^{(t)}$, the rank stores it to every other rank:

$$C_{q,r}^{(t)} \leftarrow C_r^{(t)} \quad \forall q \neq r.$$

A minimal scatter loop is:

```
for remote_rank in range(world_size):
    if remote_rank != rank:
        iris.store(
            c_tile_ptr,
            c_tile,
            from_rank=rank,
            to_rank=remote_rank,
            heap_bases=heap_bases
        )
```

The check avoids redundantly storing to local memory when the local tile already exists.

Performance Intuition

Why Bulk-Synchronous Execution Leaves Performance on the Table

In a bulk-synchronous design, communication begins only after all GEMM tiles are complete. If tile 0 finishes early, it still waits for the slowest final tile.

The wasted opportunity is:

$$\sum_{t=0}^{T-1} (\text{time between tile } t \text{ completion and communication start}).$$

Fine-grained overlap lets communication start at:

time tile t becomes ready,

not at:

$$\max_t (\text{tile } t \text{ completion time}).$$

Why Producer-Consumer Overlap Helps

Producer-consumer overlap forms a pipeline:

compute tile 0 $\rightarrow$ communicate tile 0

while later tiles are still being computed:

compute tile 1, compute tile 2, $\dots$

A steady-state pipeline time is approximately:

$$T_{\text{pipeline}} = T_{\text{fill}} + (T - 1) \max(t_c, t_m) + T_{\text{drain}},$$

where t_c is per-tile compute time and t_m is per-tile communication time. A non-overlapped execution would be:

$$T_{\text{serial}} = T(t_c + t_m).$$

As T grows, the pipeline benefit approaches:

$$\frac{t_c + t_m}{\max(t_c, t_m)}.$$

Why Resource Partitioning Is Hard

If too few SMs are assigned to communication, the communication queue grows and overlap fails. If too many are assigned to communication, the GEMM becomes slower and communication workgroups spin.

The balance condition is:

$$N_{\text{GEMM}} \cdot r_{\text{GEMM-per-SM}} \approx N_{\text{comm}} \cdot r_{\text{comm-per-SM}}.$$

However, both rates depend on shape, dtype, tile size, memory layout, interconnect bandwidth, cache policy, and compiler scheduling.

This is why the lecture discusses several possible approaches:

- manual expert tuning;
- analytical models;
- limited autotuning for fixed shapes;
- work queues that dynamically assign tasks;
- compiler assistance in choosing partitioning.

Tail Effects

A tail effect occurs when most workgroups have finished but one or a few workgroups are still running. If a fused kernel extends the duration of a final tile, then all other resources may be idle.

Let N be the number of workgroups and T_i be the runtime of workgroup i . Kernel time is:

$$T_{\text{kernel}} = \max_i T_i.$$

The total useful work is proportional to:

$$\sum_i T_i,$$

but occupied resource time is approximately:

$$N_{\text{resident}} \cdot T_{\text{kernel}}.$$

The tail waste is:

$$T_{\text{waste}} = N_{\text{resident}} \cdot \max_i T_i - \sum_i T_i.$$

Sequential fusion can increase T_i for some workgroups and therefore worsen tail waste.

Register Pressure

When GEMM and communication are fused, accumulator values may remain live while communication code executes. This increases register lifetime.

Higher register pressure can reduce occupancy:

$$\text{occupancy} \propto \frac{\text{available registers per SM}}{\text{registers per workgroup}}.$$

If registers per workgroup increase too much, fewer workgroups can be resident, reducing latency hiding. Thus fusion has a trade-off:

less HBM traffic versus more register pressure and possible lower occupancy.

Why Cache Modifiers Matter

Communication data often has little producer-side reuse. If such data occupies cache, it can evict useful GEMM operands. Cache pollution can reduce effective reuse:

$$\text{useful cache capacity} = \text{total cache capacity} - \text{polluting communication lines}.$$

For producer-only output tiles, write-through or non-temporal behavior may help:

$$\text{communication store} \not\Rightarrow \text{local cache pollution}.$$

This is particularly useful when the programmer knows the access pattern better than a generic compiler or library.

Common Misconceptions

Misconception: Iris Is Just Another Collective Library

Iris is better understood as a device-side primitive layer. It can implement collectives, and the repository includes collective-like examples, but the core value is that a Triton kernel can directly perform remote loads, stores, puts, gets, and atomics.

$$\text{Iris} \neq \text{only all-reduce}.$$

Rather:

$$\text{Iris} = \text{remote memory access primitives} + \text{Triton programmability}.$$

Misconception: Remote Stores Alone Are Enough

Remote stores move data, but they do not necessarily establish visibility and ordering for a consumer. Correct producer-consumer communication requires synchronization.

The correct pattern is:

$$\text{write data} \rightarrow \text{release flag} \rightarrow \text{acquire flag} \rightarrow \text{read data}.$$

Misconception: More Communication SMs Always Improves Performance

Adding more SMs to communication helps only until the interconnect is saturated or until compute becomes starved. The optimal split depends on the relative rates of production and consumption.

$$N_{\text{comm}} \uparrow \not\Rightarrow T_{\text{total}} \downarrow \text{ always.}$$

Misconception: Fusion Is Always Faster

Fusion can avoid global-memory round trips and reduce launch overhead, but it may increase register pressure, reduce occupancy, complicate scheduling, and worsen tail effects.

fusion benefit = saved memory traffic—extra register pressure—tail overhead—scheduling imbalance.

Misconception: PyTorch-Like Allocation Means Ordinary PyTorch Semantics Are Always Safe

Iris tensors are PyTorch tensors, but remote access depends on symmetric-heap placement. Operations that allocate new tensors can move data outside the symmetric heap.

The safe rule is:

use in-place operations when remote addressability must be preserved.

Misconception: Small Dtypes Are Conceptually Different for Communication

At the functionality level, remote communication is still a load or store regardless of dtype. If Triton can generate the relevant load or store instruction, Iris can conceptually move that data. However, performance for very small dtypes may require packing, alignment, and vectorization.

functional support $\neq$ optimal bandwidth.

Practical Takeaways

1. Iris makes multi-GPU communication a device-side Triton programming problem rather than only a host-side collective problem.

2. The symmetric heap is the key abstraction: all remote pointer translation is based on heap base plus offset.
3. The core translation equation is:

$$p_{\text{remote}} = H_{\text{remote}} + (p_{\text{local}} - H_{\text{local}}).$$

4. Correct fine-grained communication needs atomics with acquire-release semantics.
5. System scope is required for cross-GPU synchronization inside a node.
6. Communication and compute overlap is the primary performance reason to use Iris.
7. There are multiple implementation patterns: bulk synchronous, separate-kernel producer-consumer, sequential fusion, and workgroup-specialized fusion.
8. No single pattern wins universally; the best design depends on shape, dtype, interconnect, tile schedule, register pressure, and cache behavior.
9. Cache modifiers are powerful because communication data often has different locality from compute data.
10. Iris is especially valuable for expert GPU programmers who want to rapidly prototype custom multi-GPU kernels.

Project or Experiment Ideas

Experiment 1: Remote Load and Store Bandwidth

Implement microbenchmarks for local and remote memory operations. Measure:

$$B_{\text{local HBM}}, \quad B_{\text{remote XGMI}}.$$

Compare achieved bandwidth to expected peak bandwidth:

$$\eta = \frac{B_{\text{achieved}}}{B_{\text{peak}}}.$$

Vary:

- number of workgroups;
- vector width;
- loop unrolling;
- dtype;
- cache modifier.

Experiment 2: Producer-Consumer Correctness

Write a two-rank program:

1. rank 0 writes an array to rank 1;
2. rank 0 releases a flag;
3. rank 1 spins on the flag with acquire semantics;
4. rank 1 validates the data.

Then intentionally remove acquire-release semantics and test whether correctness becomes fragile.

Experiment 3: GEMM Plus All-Scatter

Implement four versions:

1. bulk-synchronous GEMM then scatter;
2. separate-kernel producer-consumer;
3. sequentially fused GEMM and scatter;
4. workgroup-specialized fused GEMM and scatter.

Measure:

$$T_{\text{GEMM}}, \quad T_{\text{comm}}, \quad T_{\text{total}}, \quad S = \frac{T_{\text{bulk}}}{T_{\text{variant}}}.$$

Experiment 4: Resource Partition Sweep

For a GPU with N_{SM} SMs, sweep:

$$N_{\text{GEMM}} \in \{32, 64, 96, \dots, N_{\text{SM}}\}.$$

Set:

$$N_{\text{comm}} = N_{\text{SM}} - N_{\text{GEMM}}.$$

Plot runtime versus N_{GEMM} and identify the best balance.

Experiment 5: Cache Modifier Study

Compare communication stores with different cache policies. Measure whether avoiding cache pollution improves GEMM throughput when compute and communication are co-resident.

Track:

L2 hit rate, HBM bandwidth, XGMI bandwidth, kernel time.

Experiment 6: Low-Precision Communication

Try FP32, FP16, BF16, FP8, and packed low-bit formats. Measure whether bandwidth is limited by:

- raw interconnect bandwidth;
- packing and unpacking overhead;
- alignment;
- vectorization;
- memory transaction granularity.

Final Mental Model

Iris turns multi-GPU programming into a natural extension of Triton programming. Instead of treating communication as a separate host-side phase, Iris lets a Triton workgroup directly reach into another GPU’s symmetric heap, perform remote loads or stores, and coordinate with atomics.

The entire system can be summarized by three equations and one synchronization rule.

The address translation equation is:

$$p_{\text{remote}} = H_{\text{remote}} + (p_{\text{local}} - H_{\text{local}}).$$

The bulk-synchronous cost model is:

$$T_{\text{bulk}} = T_{\text{compute}} + T_{\text{communication}} + T_{\text{barriers}}.$$

The overlap target is:

$$T_{\text{overlap}} \approx \max(T_{\text{compute}}, T_{\text{communication}}) + T_{\text{synchronization}}.$$

The correctness rule is:

producer writes data $\rightarrow$ release flag $\rightarrow$ consumer acquires flag $\rightarrow$ consumer reads data.

The practical mental model is:

Think of each GPU as owning a partition of a global address space. Iris gives each Triton workgroup the ability to compute a tile, translate a symmetric-heap pointer, move data directly to or from another GPU, and synchronize with precisely scoped atomics. Performance comes from overlapping tile-level communication with tile-level compute while controlling cache behavior, register pressure, resource partitioning, and memory visibility.